Arabic NLP Application

Preprocessing · Question Answering · Classification · Machine Translation

Hala Nazzal

Project Overview — Pipeline

NB 1 — Preprocessing

- Arabic text cleaning
- Tashkeel & tatweel removal
- Stopwords (NLTK)
- Alef normalization
- ISRIStemmer

NB 2 — Question Answering

- Trad(TF-IDF) /(Independent) Word2Vec
- AraBERT embeddings
- RNN, LSTM, GRU, BiGRU
- AraGPT2, mT5, Qwen

NB 3 — Classification

- TF-IDF N-gram features
- Naive Bayes, SVM, LR, RF
- BERT fine-tuning
- Best: LinearSVC 73.5%

NB 4 — Machine Translation

- OpenRouter API
- GPT-3.5-turbo model
- Arabic to English
- Category-context prompts

Activity 1 — Preprocessing Pipeline

Preprocessing Steps

1. Remove non-Arabic characters (regex)
2. Strip tashkeel — pyarabic library
3. Remove tatweel (elongation)
4. Normalize Alef variants (إ أ آ → ا)
5. Remove stopwords (NLTK Arabic corpus)
6. ISRIStemmer (words ≥ 4 chars)
7. Save processed dataset to CSV

Packages Used

pyarabic	Arabic text cleaning (tashkeel, tatweel)
NLTK	Arabic stopwords + ISRIStemmer
re	Regex for non-Arabic removal
pandas	Dataset loading and CSV export

AAFAQ Dataset Stats

Original size: 5,009 Q&A pairs

After preprocessing: 4,522 pairs

Categories: 16 topic classes

Language: Modern Standard Arabic

Word Representation Techniques — Comparison

TF-IDF
Traditional

- Sparse vector representation
- N-gram range (1,2)
- max_features = 10,000
- sublinear_tf = True
- Fast & interpretable
- Best for classification

Word2Vec
Traditional — Embedding

- Dense word embeddings
- Trained from scratch on AAFAQ
- Captures semantic similarity
- window=5, size=100
- Used in QA retrieval
- Unsupervised learning

AraBERT
Transformer — Contextual

- aubmindlab/bert-base-arabertv02
- Contextual word embeddings
- CLS token = sentence vector
- max_length = 64 tokens
- Best QA retrieval results
- Pre-trained on Arabic text

Activity 2 — Question Answering: Models & Results

Sequential Neural Models — All 3 Representations

Model	Traditional	Word2Vec	AraBERT
RNN	0.5204 ✓	0.1624	0.4464
LSTM	0.1558	0.1558	0.3171
GRU	0.1558	0.1558	0.5138 ✓
BiGRU ★	**0.6033 ★**	0.2608	0.0740

Key Findings

- BiGRU+TF-IDF best (0.60): reads forward+backward, n-gram TF-IDF captures Arabic morphology
- Word2Vec underperforms (≤0.26): AAFAQ too small (~3,600 samples) to train good embeddings
- AraBERT helps GRU (0.51) but hurts BiGRU (0.07): contextual 768-dim may overfit complex arch
- Retrieval QA (cosine sim 0.70) chosen for deployment: always grounded, no hallucination

QA Design Choices

- 128 hidden units: balance capacity vs. overfitting on small dataset
- Mean pooling (.mean(dim=1)): aggregates all token states for retrieval (CLS used in classification)
- BiGRU over GRU: reads both directions — captures context from both ends of the question

Generative Transformer Models

AraGPT2

GPT-based Arabic generation

aubmindlab/aragpt2-base

mT5-small

Sequence-to-sequence

google/mt5-small

Qwen2.5-0.5B

Large language model

Qwen/Qwen2.5-0.5B

AraBERT + Cosine Retrieval

Avg. cosine similarity: 0.70 | Retrieval-based (no hallucination)

✓ Chosen for NLP_Interface deployment

Activity 3 — Classification: Traditional vs Neural

Model Performance Comparison

Model	Type	Features	Accuracy
Naive Bayes	Traditional	TF-IDF (1,2)	0.6497
Random Forest	Traditional	TF-IDF (1,2)	0.6541
Log. Regression	Traditional	TF-IDF (1,2)	0.7116
BERT Fine-tuned	Neural (Transformer)	AraBERT embeds	0.6387
LinearSVC	Traditional	TF-IDF (1,2)	**0.7348**

Key Findings

- LinearSVC (73.5%) outperforms BERT (63.9%)
- TF-IDF bigrams capture Arabic morphology
- BERT limited by dataset size (4,522 rows)
- class_weight='balanced' handles imbalance

Design Choices

- RandomOverSampler before Naive Bayes
- ngram_range=(1,2) for phrase features
- C=1.0, max_iter=2000 for LinearSVC
- sublinear_tf=True reduces frequency bias

Activity 4 — Machine Translation (Arabic → English)

API-Based Translation Approach

API Provider	OpenRouter (openrouter.ai)
Model	openai/gpt-3.5-turbo
Method	Chat completions endpoint
Input	Arabic question + answer
Output	Natural English translation
Prompt Style	Category-context modification

Prompt Engineering

System: "You are a professional Arabic to English translator specializing in {category} topics."

User: "Translate the following Arabic text: {arabic_text}"

Translation Example

```
Arabic  : ما هي أسباب الحرب العالمية الأولى؟
English : What are the causes of World War I?

Arabic  : يعاني المريض من ارتفاع ضغط الدم
English : The patient suffers from high blood pressure.

Arabic  : أنا أدرس الذكاء الاصطناعي في الجامعة
English : I study artificial intelligence at the university.

Arabic  : كيف يمكنني تحسين مهاراتي في البرمجة؟
English : How can I improve my programming skills?

Arabic  : ما هو الاقتصاد الدائري؟
English : What is the circular economy?
```

Category context improves translation domain accuracy

Activity 2 — NLP Interface (NLP_Interface.py)

Interface Pipeline

1	Accept Arabic question — Streamlit UI
2	Apply preprocessing (same as Notebook 1)
3	QA — AraBERT cosine similarity retrieval
4	Classification — LinearSVC + TF-IDF
5	Translation — OpenRouter GPT-3.5 API
6	Display: Answer + Category + English

Technical Stack

Streamlit	Web UI framework — Python
AraBERT	bert-base-arabertv02 embeddings
LinearSVC	scikit-learn, accuracy 73.5%
TF-IDF	ngram_range=(1,2), 10K features
OpenRouter	GPT-3.5-turbo translation
pyarabic	Arabic text preprocessing

Conclusions & System Evaluation

Preprocessing

- pyarabic most effective for Arabic-specific cleaning
- ISRIStemmer reduces vocab while preserving meaning
- Alef normalization reduces vocabulary fragmentation
- Evaluation: NLTK stopwords most impactful for quality

Model Comparison

- LinearSVC (73.5%) > BERT (63.9%) for classification
- BiGRU (60.3%) best sequential model for QA
- AraBERT embeddings best for semantic QA retrieval
- Traditional models competitive with neural on small data

System Design

- API translation = scalable and accurate
- st.cache_resource = fast repeated interface use
- TF-IDF bigrams handle Arabic morphology effectively
- Category-context prompts improve translation quality

Overall: The Arabic NLP pipeline successfully integrates preprocessing, retrieval-based QA, multi-model classification, API translation, and a Streamlit interface into one cohesive deployment-ready application.